Functions and tidy evaluation

Based on Chapter 25 from *R for Data Science*

You can download this .qmd file from here. Just hit the Download Raw File button.

Introduction (from Ch 25 of R4DS)

One of the best ways to improve your reach as a data scientist is to write functions. Functions allow you to automate common tasks in a more powerful and general way than copy-and-pasting. Writing a function has four big advantages over using copy-and-paste:

- You can give a function an evocative name that makes your code easier to understand.
- As requirements change, you only need to update code in one place, instead of many.
- You eliminate the chance of making incidental mistakes when you copy and paste (i.e. updating a variable name in one place, but not in another).
- It makes it easier to reuse work from project-to-project, increasing your productivity over time.

A good rule of thumb is to consider writing a function whenever you've copied and pasted a block of code more than twice (i.e. you now have three copies of the same code). We'll learn about three useful types of functions:

- Vector functions take one or more vectors as input and return a vector as output.
- Data frame functions take a data frame as input and return a data frame as output.
- Plot functions that take a data frame as input and return a plot as output.

```
# Initial packages required (we'll be adding more)
library(tidyverse)
library(nycflights13)
```

Do not Repeat Yourself: Also known as DRY, if you copy or paste code more than twice, you should write a function instead.

When writing a function, it is usually best to start with the code you know works for one instance, and then "function-ize" it.

Vector functions

Example 1: Rescale variables from 0 to 1.

This code creates a 10 x 4 tibble filled with random values taken from a normal distribution with mean 0 and SD 1

```
df <- tibble(
  a = rnorm(10),
  b = rnorm(10),
  c = rnorm(10),
  d = rnorm(10)
)
df
```

```
# A tibble: 10 x 4
        a      b        c      d
    <dbl>  <dbl>    <dbl>  <dbl>
 1 -0.140  0.406  0.281    0.771
 2  0.959 -0.650 -0.0288  -0.548
 3 -1.02  -0.474 -0.0484   0.137
 4 -1.38  -1.16   1.32    -0.150
 5  0.820  0.462 -0.831   -1.15
 6  1.68   0.244 -1.36     0.586
 7 -0.529 -0.886 -1.56    -0.134
 8 -1.47  -1.50  -1.24     0.935
 9  1.78  -0.369 -0.0311   0.777
10 -0.763  0.939  0.00998 -0.495
```

This code below for rescaling variables from 0 to 1 is ripe for functions… we did it four times!

It's easiest to start with working code and turn it into a function.

```
df$a <- (df$a - min(df$a)) / (max(df$a) - min(df$a))
df$b <- (df$b - min(df$b)) / (max(df$b) - min(df$b))
df$c <- (df$c - min(df$c)) / (max(df$c) - min(df$c))
df$d <- (df$d - min(df$d)) / (max(df$d) - min(df$d))
df
```

```
# A tibble: 10 x 4
        a     b      c     d
    <dbl> <dbl>  <dbl> <dbl>
```

```
 1 0.408  0.782 0.638  0.922
 2 0.746  0.350 0.531  0.289
 3 0.137  0.422 0.524  0.617
 4 0.0266 0.143 1      0.479
 5 0.703  0.805 0.252  0
 6 0.969  0.715 0.0686 0.833
 7 0.289  0.253 0      0.487
 8 0      0     0.111  1
 9 1      0.465 0.530  0.925
10 0.217  1     0.544  0.314
```

Notice first what changes and what stays the same in each line. Then, if we look at the first line above, we see we have one value we're using over and over: `df$a`. So our function will have one input. We'll start with our code from that line, then replace the input (df$a) with x. We should give our function a name that explains what it does. The name should be a verb.

```
# I'm going to show you how to write the function in class!
# I have it in the code already below, but don't look yet!
# Let's try to write it together first!
```

. . . . . .

```
# Our function (first draft!)
rescale01 <- function(x) {
  (x - min(x)) / (max(x) - min(x))
}
```

Note the **general form of a function**:

```
name <- function(arguments) {
  body
}
```

Every function contains 3 essential components:

1. A **name**. The name should clearly evoke what the function does; hence, it is often a verb (action). Here we'll use rescale01 because this function rescales a vector to lie between 0 and 1. snake_case is good; CamelCase is just okay.
2. The **arguments**. The arguments are things that vary across calls and they are usually nouns - first the data, then other details. Our analysis above tells us that we have just one; we'll call it x because this is the conventional name for a numeric vector, but you can use any word.

3. The **body**. The body is the code that's repeated across all the calls. By default a function will return the last statement; use `return()` to specify a return value

   Summary: Functions should be written for both humans and computers!

Once we have written a function we like, then we need to test it with different inputs!

```
temp <- c(4, 6, 8, 9)
rescale01(temp)
```

```
[1] 0.0 0.4 0.8 1.0
```

```
temp0 <- c(4, 6, 8, 9, NA)
rescale01(temp0)
```

```
[1] NA NA NA NA NA
```

OK, so NA's don't work the way we want them to.

```
rescale01 <- function(x) {
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
}
rescale01(temp)
```

```
[1] 0.0 0.4 0.8 1.0
```

```
rescale01(temp0)
```

```
[1] 0.0 0.4 0.8 1.0  NA
```

We can continue to improve our function. Here is another method, which uses the existing `range` function within R to avoid 3 max/min executions:

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01(temp)
```

```
[1] 0.0 0.4 0.8 1.0
```

```
rescale01(c(0, 5, 10))
```

```
[1] 0.0 0.5 1.0
```

```
rescale01(c(-10, 0, 10))
```

```
[1] 0.0 0.5 1.0
```

```
rescale01(c(1, 2, 3, NA, 5))
```

```
[1] 0.00 0.25 0.50   NA 1.00
```

We should continue testing unusual inputs. Think carefully about how you want this function to behave… the current behavior is to include the Inf (infinity) value when calculating the range. You get strange output everywhere, but it's pretty clear that there is a problem right away when you use the function. In the example below (rescale1), you ignore the infinity value when calculating the range. The function returns Inf for one value, and sensible stuff for the rest. In many cases this may be useful, but it could also hide a problem until you get deeper into an analysis.

```
x <- c(1:10, Inf)
rescale01(x)
```

```
 [1]   0   0   0   0   0   0   0   0   0   0 NaN
```

```
rescale1 <- function(x) {
  rng <- range(x, na.rm = TRUE, finite = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale1(x)
```

```
 [1] 0.0000000 0.1111111 0.2222222 0.3333333 0.4444444 0.5555556 0.6666667
 [8] 0.7777778 0.8888889 1.0000000       Inf
```

Now we've used functions to simplify original example. We will learn to simplify further in iterations (Ch 26)

```r
df <- tibble(
  a = rnorm(10),
  b = rnorm(10),
  c = rnorm(10),
  d = rnorm(10)
)
# add a little noise
df$a[5] = NA
df$b[6] = Inf
df
```

```
# A tibble: 10 x 4
        a        b      c        d
    <dbl>    <dbl>  <dbl>    <dbl>
 1 -0.689  -0.0662  0.254  1.04
 2 -0.429  -0.387   0.151 -1.38
 3  0.915   0.331   0.446  0.00207
 4  1.02    1.41   -2.11  -0.0590
 5 NA      -0.576   0.669 -0.651
 6 -0.763 Inf      -1.79  -1.44
 7 -1.94   -0.814  -0.133  1.66
 8  1.23    0.307   0.874 -0.0548
 9 -0.535   0.505   0.175  0.160
10 -0.226   0.103   0.436 -0.884
```

```r
df$a_new <- rescale1(df$a)
df$b_new <- rescale1(df$b)
df$c_new <- rescale1(df$c)
df$d_new <- rescale1(df$d)
df
```

```
# A tibble: 10 x 8
        a        b      c        d  a_new   b_new c_new  d_new
    <dbl>    <dbl>  <dbl>    <dbl>  <dbl>   <dbl> <dbl>  <dbl>
 1 -0.689  -0.0662  0.254  1.04     0.395   0.336 0.792 0.801
 2 -0.429  -0.387   0.151 -1.38     0.477   0.192 0.758 0.0204
 3  0.915   0.331   0.446  0.00207  0.902   0.515 0.857 0.466
 4  1.02    1.41   -2.11  -0.0590   0.935   1     0     0.446
 5 NA      -0.576   0.669 -0.651   NA       0.107 0.931 0.255
 6 -0.763 Inf      -1.79  -1.44     0.372 Inf     0.107 0
 7 -1.94   -0.814  -0.133  1.66     0       0     0.662 1
```

```
 8  1.23    0.307   0.874 -0.0548   1       0.504 1     0.448
 9 -0.535   0.505   0.175  0.160    0.444   0.593 0.766 0.517
10 -0.226   0.103   0.436 -0.884    0.541   0.412 0.853 0.180
```

```
df |>
  select(1:4) |>
  mutate(a_new = rescale1(a),
         b_new = rescale1(b),
         c_new = rescale1(c),
         d_new = rescale1(d))
```

```
# A tibble: 10 x 8
        a        b      c        d  a_new   b_new c_new  d_new
    <dbl>    <dbl>  <dbl>    <dbl>  <dbl>   <dbl> <dbl>  <dbl>
 1 -0.689  -0.0662  0.254  1.04     0.395   0.336 0.792 0.801
 2 -0.429  -0.387   0.151 -1.38     0.477   0.192 0.758 0.0204
 3  0.915   0.331   0.446  0.00207  0.902   0.515 0.857 0.466
 4  1.02    1.41   -2.11  -0.0590   0.935   1     0     0.446
 5 NA      -0.576   0.669 -0.651   NA       0.107 0.931 0.255
 6 -0.763 Inf      -1.79  -1.44     0.372 Inf     0.107 0
 7 -1.94   -0.814  -0.133  1.66     0       0     0.662 1
 8  1.23    0.307   0.874 -0.0548   1       0.504 1     0.448
 9 -0.535   0.505   0.175  0.160    0.444   0.593 0.766 0.517
10 -0.226   0.103   0.436 -0.884    0.541   0.412 0.853 0.180
```

```
# Even better - from Chapter 26
df |>
  select(1:4) |>
  mutate(across(a:d, rescale1, .names = "{.col}_new"))
```

```
# A tibble: 10 x 8
        a        b      c        d  a_new   b_new c_new  d_new
    <dbl>    <dbl>  <dbl>    <dbl>  <dbl>   <dbl> <dbl>  <dbl>
 1 -0.689  -0.0662  0.254  1.04     0.395   0.336 0.792 0.801
 2 -0.429  -0.387   0.151 -1.38     0.477   0.192 0.758 0.0204
 3  0.915   0.331   0.446  0.00207  0.902   0.515 0.857 0.466
 4  1.02    1.41   -2.11  -0.0590   0.935   1     0     0.446
 5 NA      -0.576   0.669 -0.651   NA       0.107 0.931 0.255
 6 -0.763 Inf      -1.79  -1.44     0.372 Inf     0.107 0
 7 -1.94   -0.814  -0.133  1.66     0       0     0.662 1
 8  1.23    0.307   0.874 -0.0548   1       0.504 1     0.448
```

```
 9 -0.535   0.505   0.175  0.160    0.444   0.593 0.766 0.517
10 -0.226   0.103   0.436 -0.884    0.541   0.412 0.853 0.180
```

Options for handling NAs in functions

Before we try some practice problems, let's consider various options for handling NAs in functions. We used the `na.rm` option within functions like `min`, `max`, and `range` in order to take care of missing values. But there are alternative approaches:

- filter/remove the NA values before rescaling
- create an if statement to check if there are NAs; return an error if NAs exist
- create a removeNAs option in the function we are creating

Let's take a look at each alternative approach in turn:

Filter/remove the NA values before rescaling

```
df <- tibble(
  a = rnorm(10),
  b = rnorm(10),
  c = rnorm(10),
  d = rnorm(10)
)
df$a[5] = NA
df
```

```
# A tibble: 10 x 4
         a       b      c      d
     <dbl>   <dbl>  <dbl>  <dbl>
 1  1.56    0.450  -0.651  1.49
 2  0.0833  0.0114 -1.46   0.336
 3  0.141  -0.312   0.171 -0.589
 4 -0.246   0.638   0.461  0.492
 5 NA       0.517  -0.760  1.10
 6  0.630   0.831   1.12   1.15
 7 -1.19   -1.21   -0.610  0.543
 8  0.101  -0.238   0.989  1.16
 9  0.324   1.44    0.997 -0.283
10  0.283   0.341  -2.12   0.601
```

```r
rescale_basic <- function(x) {
  (x - min(x)) / (max(x) - min(x))
}

df |>
  filter(!is.na(a)) |>
  mutate(new_a = rescale_basic(a))
```

```
# A tibble: 9 x 5
        a       b      c      d new_a
    <dbl>   <dbl>  <dbl>  <dbl> <dbl>
1  1.56    0.450  -0.651  1.49  1
2  0.0833  0.0114 -1.46   0.336 0.463
3  0.141  -0.312   0.171 -0.589 0.484
4 -0.246   0.638   0.461  0.492 0.344
5  0.630   0.831   1.12   1.15  0.662
6 -1.19   -1.21   -0.610  0.543 0
7  0.101  -0.238   0.989  1.16  0.470
8  0.324   1.44    0.997 -0.283 0.551
9  0.283   0.341  -2.12   0.601 0.536
```

[Pause to Ponder:] Do you notice anything in the output above that gives you pause?

The row in the data frame with an NA value is completely dropped. Now the data frame only has 9 rows after using the function.

Create an if statement to check if there are NAs; return an error if NAs exist

First, here's an example involving weighted means:

```r
# Create function to calculate weighted mean
wt_mean <- function(x, w) {
  sum(x * w) / sum(w)
}
wt_mean(c(1, 10), c(1/3, 2/3))
```

```
[1] 7
```

```r
wt_mean(1:6, 1:3)
```

```
[1] 7.666667
```

[**Pause to Ponder:**] Why is the answer to the last call above 7.67? Aren't we taking a weighted mean of 1-6, all of which are below 7?

The weighted mean is greater than 7 because the lengths of x and w differ. Because of this, each entry 1 through 6 is multiplied by a value of 1:3 going in order. Entries 4 5 and 6 are multiplied by 1 2 and 3 respectively, as shown in the output below which makes the weighted mean greater than 7.

```
(1:6)*(1:3)
```

```
[1]  1  4  9  4 10 18
```

```
# update function to handle cases where data and weights of unequal length
wt_mean <- function(x, w) {
  if (length(x) != length(w)) {
    stop("`x` and `w` must be the same length", call. = FALSE)
  } else {
  sum(w * x) / sum(w)
  }
}
wt_mean(1:6, 1:3)
```

```
Error:
! `x` and `w` must be the same length
```

```
# should produce an error now if weights and data different lengths
#  - nice example of if and else
```

[**Pause to Ponder:**] What does the `call.` option do?

call. = FALSE removes the source of the error in the output to only leave the message that is specified in stop.

Now let's apply this to our rescaling function

```
rescale_w_error <- function(x) {
  if (is.na(sum(x))) {
    stop("`x` cannot have NAs", call. = FALSE)
  } else {
    (x - min(x)) / (max(x) - min(x))
  }
}
```

```
temp <- c(4, 6, 8, 9)
rescale_w_error(temp)
```

```
[1] 0.0 0.4 0.8 1.0
```

```
temp <- c(4, 6, 8, 9, NA)
rescale_w_error(temp)
```

```
Error:
! `x` cannot have NAs
```

[Pause to Ponder:] Why can't we just use `if (is.na(x))` instead of `is.na(sum(x))`?

We have to use sum within the is.na because the is.na returns an output for each entry in the vector rather than a logical statement that is true or false. Within if, there needs to be a logical statement and determining whether the sum of x is na provides that whereas only including is.na does not specify a condition.

Create a removeNAs option in the function we are creating

```
rescale_NAoption <- function(x, removeNAs = FALSE) {
  (x - min(x, na.rm = removeNAs)) /
    (max(x, na.rm = removeNAs) - min(x, na.rm = removeNAs))
}

temp <- c(4, 6, 8, 9)
rescale_NAoption(temp)
```

```
[1] 0.0 0.4 0.8 1.0
```

```
temp <- c(4, 6, 8, 9, NA)
rescale_NAoption(temp, removeNAs = TRUE)
```

```
[1] 0.0 0.4 0.8 1.0  NA
```

OK, but all the other summary stats functions use na.rm as the input, so to be consistent, it's probably better to do something slightly awkward like this:

```r
rescale_NAoption <- function(x, na.rm = FALSE) {
  (x - min(x, na.rm = na.rm)) /
    (max(x, na.rm = na.rm) - min(x, na.rm = na.rm))
}

temp <- c(4, 6, 8, 9, NA)
rescale_NAoption(temp, na.rm = TRUE)
```

```
[1] 0.0 0.4 0.8 1.0  NA
```

`wt_mean()` is an example of a "summary function (single value output) instead of a"mutate function" (vector output) like `rescale01()`. Here's another summary function to produce the mean absolute percentage error:

```r
mape <- function(actual, predicted) {
  sum(abs((actual - predicted) / actual)) / length(actual)
}

y <- c(2,6,3,8,5)
yhat <- c(2.5, 5.1, 4.4, 7.8, 6.1)
mape(actual = y, predicted = yhat)
```

```
[1] 0.2223333
```

Data frame functions

These work like dplyr verbs, taking a data frame as the first argument, and then returning a data frame or a vector.

Demonstration of tidy evaluation in functions

```r
# Start with working code then functionize
ggplot(data = mpg, mapping = aes(x = mpg$cty, y = mpg$hwy)) +
  geom_point(size = 0.75) +
  geom_smooth()
```

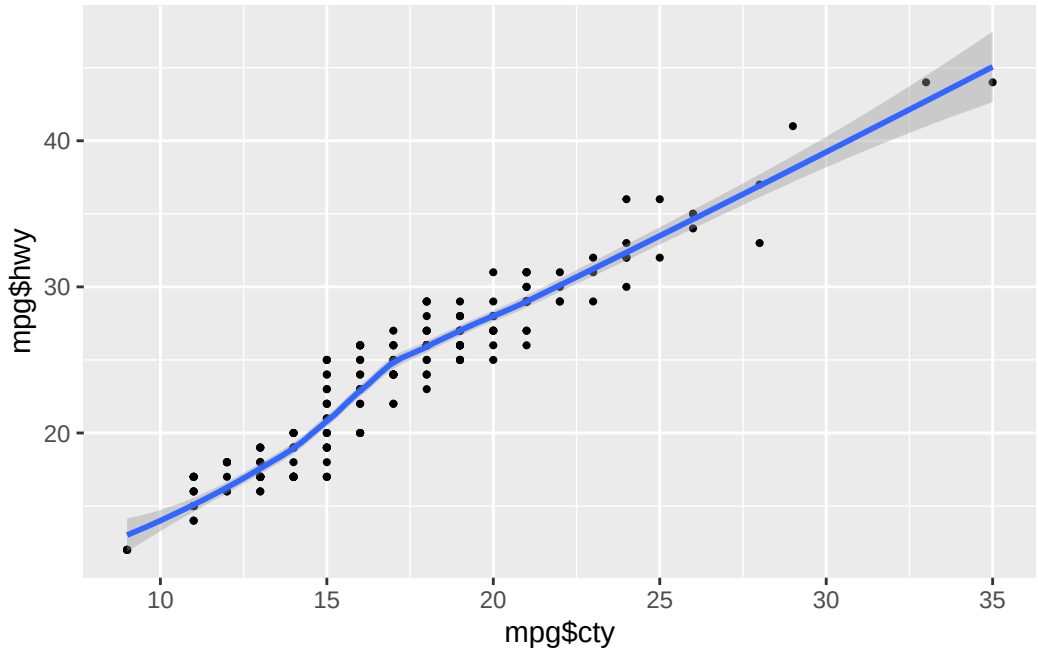
```
Warning: Use of `mpg$cty` is discouraged.
i Use `cty` instead.
```

```
Warning: Use of `mpg$hwy` is discouraged.
i Use `hwy` instead.
```

```
Warning: Use of `mpg$cty` is discouraged.
i Use `cty` instead.
```

```
Warning: Use of `mpg$hwy` is discouraged.
i Use `hwy` instead.
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
make_plot <- function(dataset, xvar, yvar, pt_size = 0.75)  {
  ggplot(data = dataset, mapping = aes(x = xvar, y = yvar)) +
    geom_point(size = pt_size) +
    geom_smooth()
}

make_plot(dataset = mpg, xvar = cty, yvar = hwy)  # Error!
```

```
Error in `geom_point()`:
! Problem while computing aesthetics.
```

```
i Error occurred in the 1st layer.
Caused by error:
! object 'cty' not found
```

The problem is tidy evaluation, which makes most common coding easier, but makes some less common things harder. Key terms to understand tidy evaluation:

- env-variables = live in the environment (mpg)
- data-variables = live in data frame or tibble (cty)
- data masking = tidyverse use data-variables as if they are env-variables. That is, you don't always need `mpg$cty` to access `cty` in tidyverse

The key idea behind data masking is that it blurs the line between the two different meanings of the word "variable":

- env-variables are "programming" variables that live in an environment. They are usually created with <-.
- data-variables are "statistical" variables that live in a data frame. They usually come from data files (e.g. .csv, .xls), or are created manipulating existing variables.

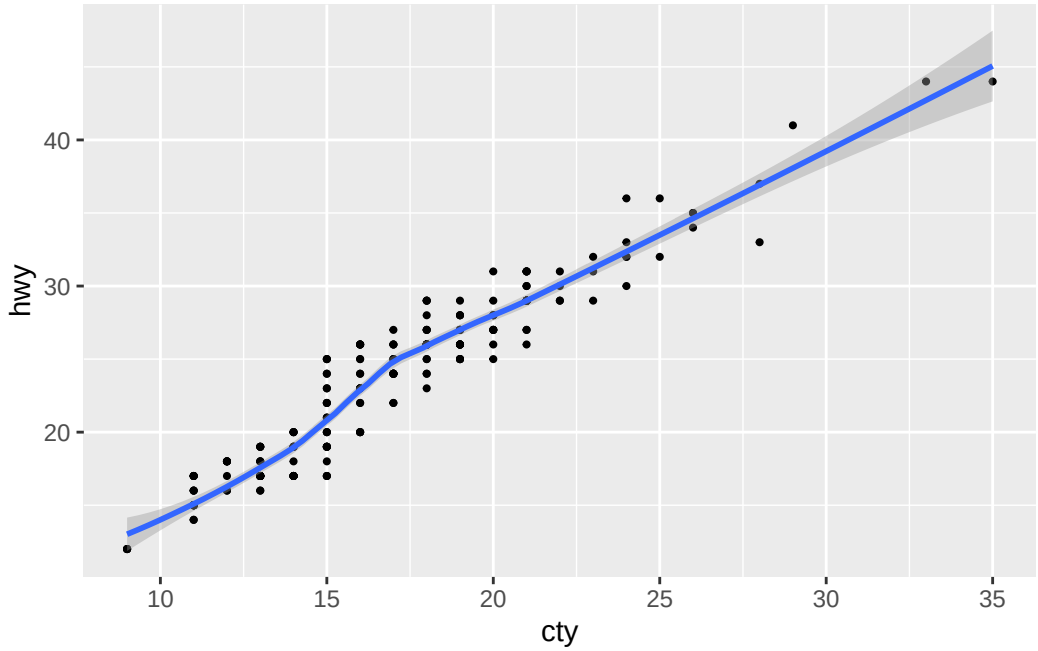
The solution is to embrace {{ }} data-variables which are user inputs into functions. One way to remember what's happening, as suggested by our book authors, is to think of {{ }} as looking down a tunnel — {{ var }} will make a dplyr function look inside of `var` rather than looking for a variable called `var`. Thus, embracing a variable tells dplyr to use the value stored inside the argument, not the argument as the literal variable name.

See Section 25.3 of R4DS for more details (and there are plenty!).

```
# This will work to make our plot!
make_plot <- function(dataset, xvar, yvar, pt_size = 0.75)  {
  ggplot(data = dataset, mapping = aes(x = {{ xvar }}, y = {{ yvar }})) +
    geom_point(size = pt_size) +
    geom_smooth()
}

make_plot(dataset = mpg, xvar = cty, yvar = hwy)
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



I often wish it were easier to get my own custom summary statistics for numeric variables in EDA rather than using `mosaic::favstats()`. Using `group_by()` and `summarise()` from the tidyverse reads clearly but takes so many lines, but if I only had to write the code once…

```
summary6 <- function(data, var) {
  data |> summarize(
    mean = mean({{ var }}, na.rm = TRUE),
    median = median({{ var }}, na.rm = TRUE),
    sd = sd({{ var }}, na.rm = TRUE),
    IQR = IQR({{ var }}, na.rm = TRUE),
    n = n(),
    n_miss = sum(is.na({{ var }})),
    .groups = "drop"    # to leave the data in an ungrouped state
  )
}

mpg |> summary6(hwy)
```

```
# A tibble: 1 x 6
   mean median    sd   IQR     n n_miss
  <dbl>  <dbl> <dbl> <dbl> <int>  <int>
1  23.4     24  5.95     9   234      0
```

Even cooler, I can use my new function with `group_by()`!

```
mpg |>
  group_by(drv) |>
  summary6(hwy)
```

```
# A tibble: 3 x 7
  drv    mean median    sd   IQR     n n_miss
  <chr> <dbl>  <dbl> <dbl> <dbl> <int>  <int>
1 4      19.2     18  4.08     5   103      0
2 f      28.2     28  4.21     3   106      0
3 r      21       21  3.66     7    25      0
```

You can even pass conditions into a function using the embrace:

[Pause to Ponder:] Predict what the code below will do, and (only) then run it to check. Think about: why do we have `sort = sort`? why not embrace `df`? why didn't we need `n` in the arguments?

The function uses a data frame, variable, and condition to create a new tibble of the sorted count and proportion of that count for the variable from a filtered version of the original tibble. The variable and condition are in double brackets because they are from within the greater tibble (not found in the environment). We use sort = sort because we want to use the default setting or sort in the function, which is true. We didn't use n in the arguments because it comes from the column name of the tibble resulting from the count function. The function is ran for 7 manufacturers in the mpg dataset and provides their sorted count and the proportion that that car manufacturer appears.

```
new_function <- function(df, var, condition, sort = TRUE) {
  df |>
    filter({{ condition }}) |>
    count({{ var }}, sort = sort) |>
    mutate(prop = n / sum(n))
}

mpg |> new_function(var = manufacturer,
                    condition = manufacturer %in% c("audi",
                                                    "honda",
                                                    "hyundai",
                                                    "nissan",
                                                    "subaru",
                                                    "toyota",
```

```
                                                    "volkswagen")
                    )
```

Data-masking vs. tidy-selection (Section 25.3.4)

Why doesn't the following code work?

```
count_missing <- function(df, group_vars, x_var) {
  df |>
    group_by({{ group_vars }}) |>
    summarize(
      n_miss = sum(is.na({{ x_var }})),
      .groups = "drop"
    )
}

flights |>
  count_missing(c(year, month, day), dep_time)
```

```
Error in `group_by()`:
i In argument: `c(year, month, day)`.
Caused by error:
! `c(year, month, day)` must be size 336776 or 1, not 1010328.
```

The problem is that `group_by()` uses data-masking rather than tidy-selection; it is selecting certain variables rather than evaluating values of those variables. These are the two most common subtypes of tidy evaluation:

- Data-masking is used in functions like arrange(), filter(), mutate(), and summarize() that compute with variables. Data masking is an R feature that blends programming variables that live inside environments (env-variables) with statistical variables stored in data frames (data-variables).

- Tidy-selection is used for functions like select(), relocate(), and rename() that select variables. Tidy selection provides a concise dialect of R for selecting variables based on their names or properties.

More detail can be found here.

The error above can be solved by using the `pick()` function, which uses tidy selection inside of data masking:

```
count_missing <- function(df, group_vars, x_var) {
  df |>
    group_by(pick({{ group_vars }})) |>
    summarize(
      n_miss = sum(is.na({{ x_var }})),
      .groups = "drop"
  )
}

flights |>
  count_missing(c(year, month, day), dep_time)
```

```
# A tibble: 365 x 4
    year month   day n_miss
   <int> <int> <int>  <int>
 1  2013     1     1      4
 2  2013     1     2      8
 3  2013     1     3     10
 4  2013     1     4      6
 5  2013     1     5      3
 6  2013     1     6      1
 7  2013     1     7      3
 8  2013     1     8      4
 9  2013     1     9      5
10  2013     1    10      3
# i 355 more rows
```

[**Pause to Ponder:**] Here's another nice use of `pick()`. Predict what the function will do, then run the code to see if you are correct.

The function takes rows and column inputs from a certain dataset, counts the combinations of the specified variables then pivots the counts into new columns where the value is the number of occurrences in that count. The list is sorted alphabetically by name. For the mpg data set, the result will be a tibble with manufacturer, model, and the different numbers of cylinders as columns and the values represent the counts for each cylinder for that specific car brand and model.

```
# Source: https://twitter.com/pollicipes/status/1571606508944719876
new_function <- function(data, rows, cols) {
  data |>
    count(pick(c({{ rows }}, {{ cols }}))) |>
    pivot_wider(
```

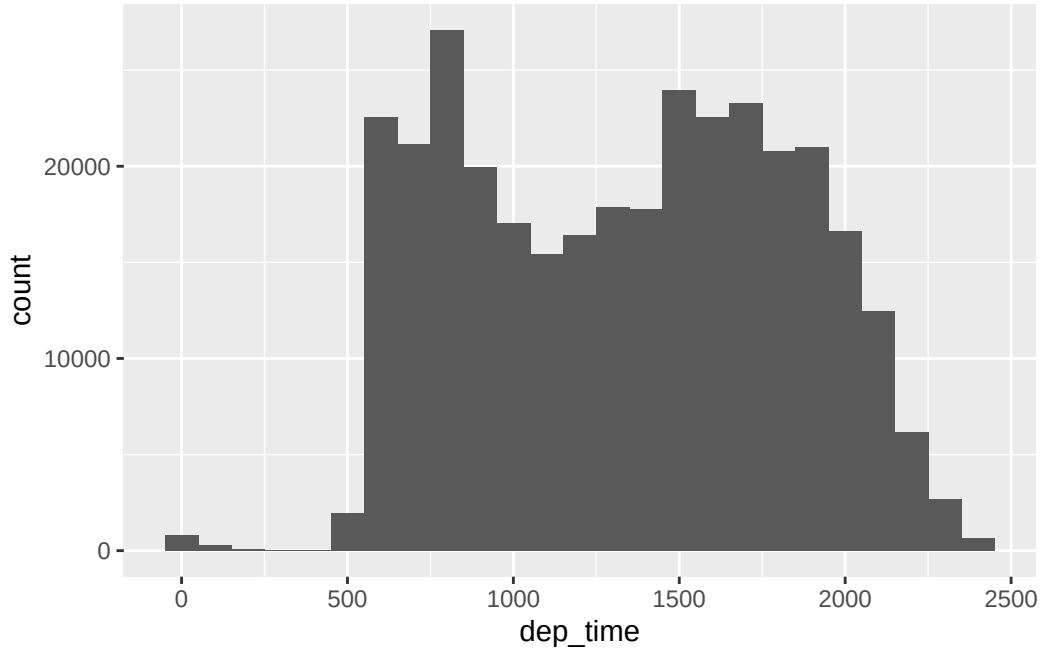
```
      names_from = {{ cols }},
      values_from = n,
      names_sort = TRUE,
      values_fill = 0
    )
}

mpg |> new_function(c(manufacturer, model), cyl)
```

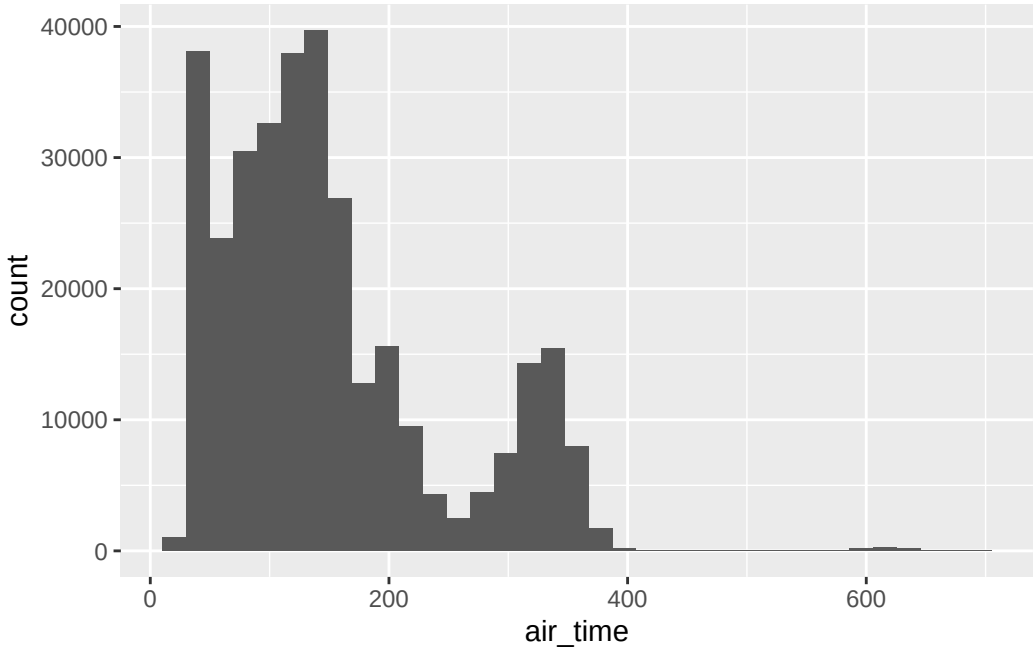
Plot functions

Let's say you find yourself making a lot of histograms:

```
flights |>
  ggplot(aes(x = dep_time)) +
  geom_histogram(bins = 25)
```



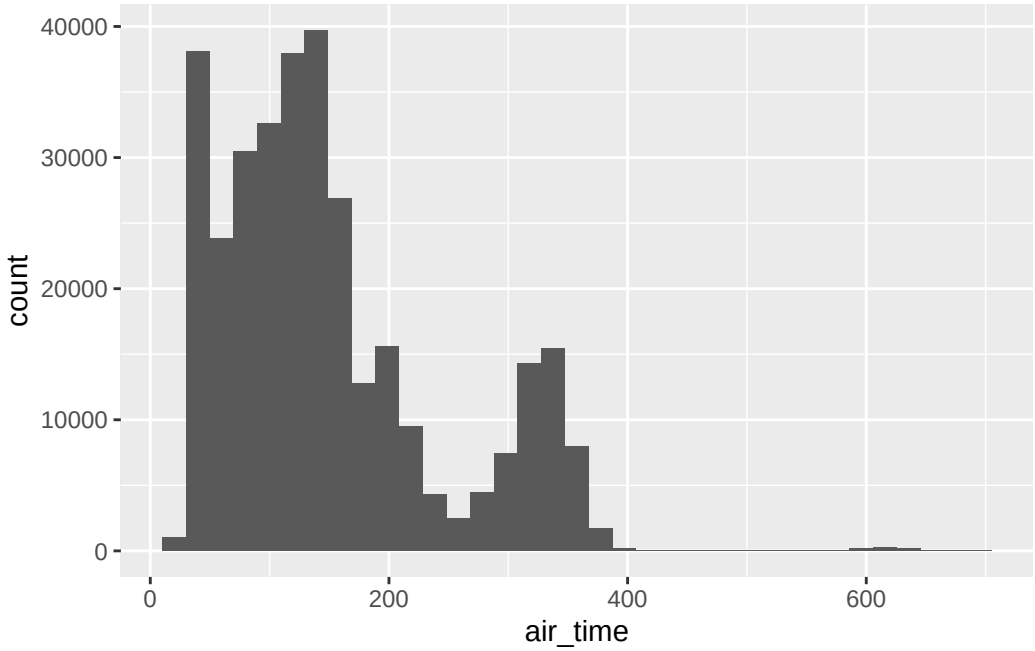

```
flights |>
  ggplot(aes(x = air_time)) +
  geom_histogram(bins = 35)
```



Just use embrace to create a histogram-making function

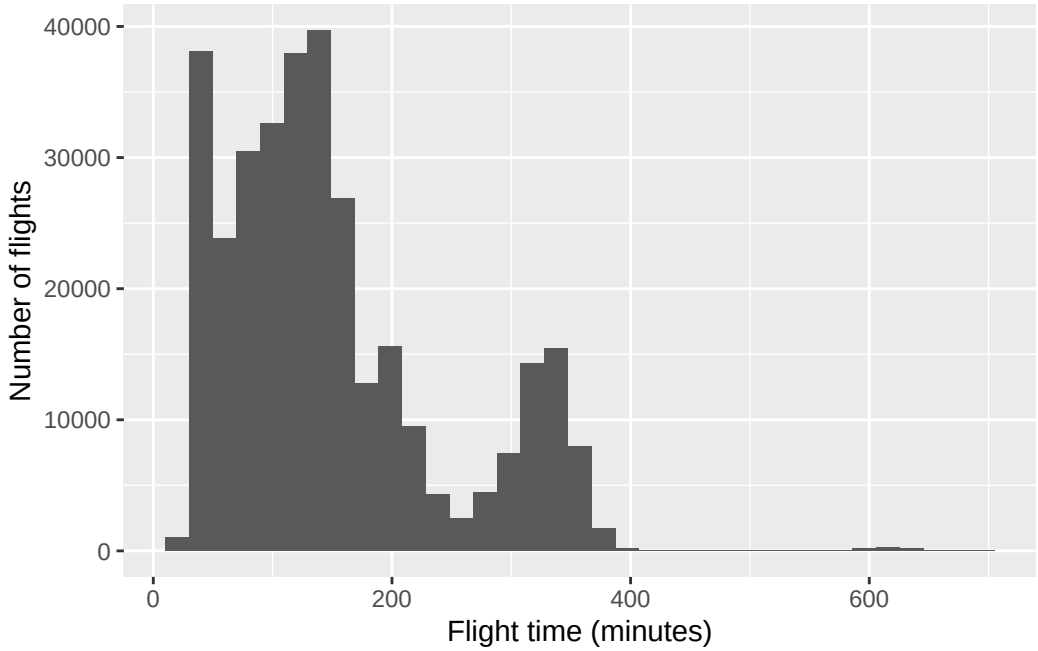
```
histogram <- function(df, var, bins = NULL) {
  df |>
    ggplot(aes(x = {{ var }})) +
    geom_histogram(bins = bins)
}

flights |> histogram(air_time, 35)
```



Since histogram() returns a ggplot, you can add any layers you want

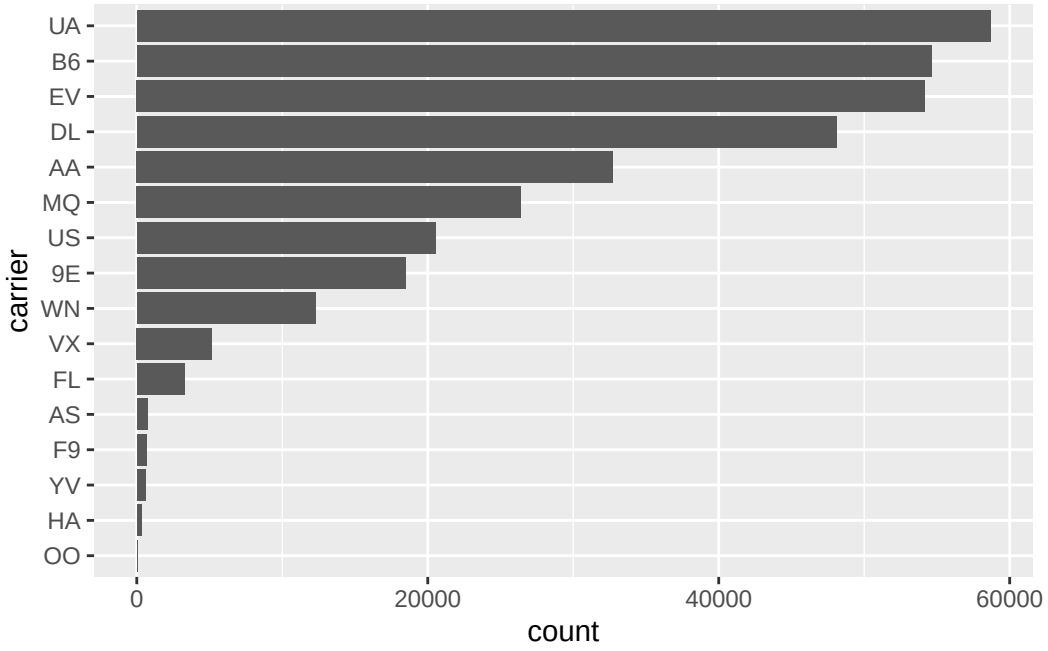
```
flights |>
  histogram(air_time, 35) +
  labs(x = "Flight time (minutes)", y = "Number of flights")
```



You can also combine data wrangling with plotting. Note that we need the "walrus operator" (:=) since the variable name on the left is being generated with user-supplied data.

```
# sort counts with highest values at top and counts on x-axis
sorted_bars <- function(df, var) {
  df |>
    mutate({{ var }} := fct_rev(fct_infreq({{ var }})))  |>
    ggplot(aes(y = {{ var }})) +
    geom_bar()
}

flights |> sorted_bars(carrier)
```



Finally, it would be really helpful to label plots based on user inputs. This is a bit more complicated, but still do-able!

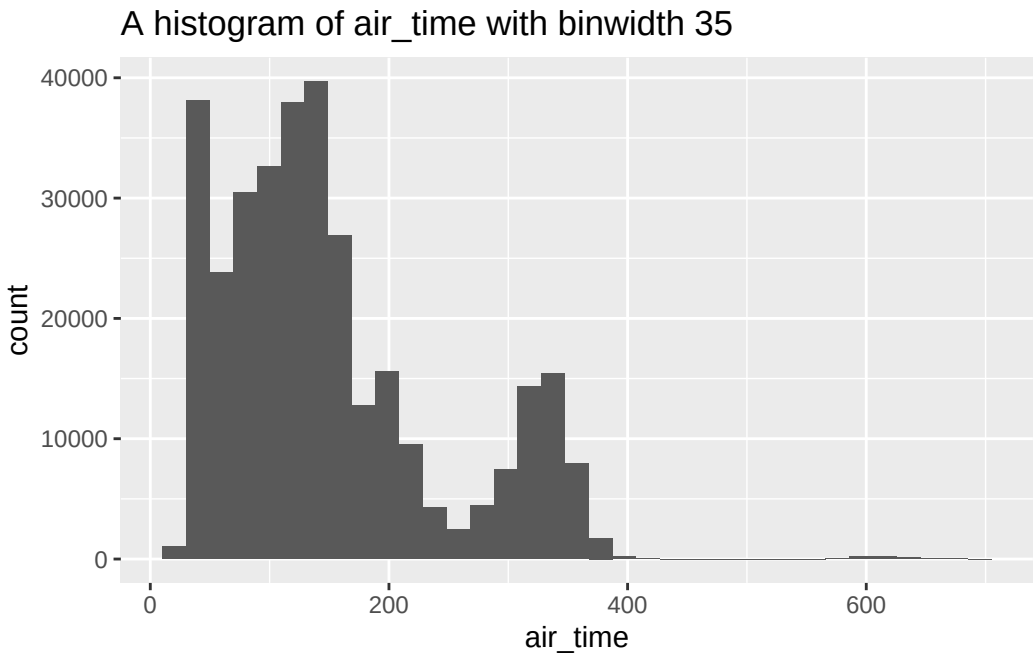
For this, we'll need the `rlang` package. `rlang` is a low-level package that's used by just about every other package in the tidyverse because it implements tidy evaluation (as well as many other useful tools).

Check out the following update of our `histogram()` function which uses the `englue()` function from the `rlang` package:

```
histogram <- function(df, var, bins) {
  label <- rlang::englue("A histogram of {{var}} with binwidth {bins}")

  df |>
    ggplot(aes(x = {{ var }})) +
    geom_histogram(bins = bins) +
    labs(title = label)
}

flights |> histogram(air_time, 35)
```



On Your Own

1. Rewrite this code snippet as a function: `x / sum(x, na.rm = TRUE)`. This code creates weights which sum to 1, where NA values are ignored. Test it for at least two different vectors. (Make sure at least one has NAs!)

```
test_x <- c(1, 2, 3, NA, NA)
make_weights <- function(x) {
  x / sum(x, na.rm = TRUE)
}



make_weights(test_x)
```

```
[1] 0.1666667 0.3333333 0.5000000        NA        NA
```

```
make_weights(x)
```

```
 [1]   0   0   0   0   0   0   0   0   0   0 NaN
```

2. Create a function to calculate the standard error of a variable, where SE = square root of the variance divided by the sample size. Hint: start with a vector like `x <- 0:5` or `x <- gss_cat$age` and write code to find the SE of x, then turn it into a function to handle any vector `x`. Note: `var` is the function to find variance in R and `sqrt` does square root. `length` may also be handy. Test your function on two vectors that do not include NAs (i.e. do **not** worry about removing NAs at this point).

```
x <- 0:5

SE <- function(x) {
sqrt(var(x)/length(x))
  }

SE(x)
```

```
[1] 0.7637626
```

3. Use your `se` function within summarize to get a table of the mean and s.e. of `hwy` and `cty` by `class` in the `mpg` dataset.

```
mpg |>
  group_by(class) |>
  summarize(mean_hwy = mean(hwy),
            se_hwy = SE(hwy),
            mean_cty = mean(cty),
            se_cty = SE(hwy))
```

```
# A tibble: 7 x 5
  class      mean_hwy se_hwy mean_cty se_cty
  <chr>         <dbl>  <dbl>    <dbl>  <dbl>
1 2seater        24.8  0.583     15.4  0.583
2 compact        28.3  0.552     20.1  0.552
3 midsize        27.3  0.334     18.8  0.334
4 minivan        22.4  0.622     15.8  0.622
5 pickup         16.9  0.396     13    0.396
6 subcompact     28.1  0.909     20.4  0.909
7 suv            18.1  0.378     13.5  0.378
```

4. Use your `se` function within summarize to get a table of the mean and s.e. of `arr_delay` and `dep_delay` by carrier in the `flights` dataset. Why does the output look like this?

```r
flights |>
  group_by(carrier) |>
  summarize(mean_arr_delay = mean(arr_delay),
            se_arr_delay = SE(arr_delay),
            mean_dep_delay = mean(dep_delay),
            se_dep_delay = SE(dep_delay))
```

```
# A tibble: 16 x 5
   carrier mean_arr_delay se_arr_delay mean_dep_delay se_dep_delay
   <chr>            <dbl>        <dbl>          <dbl>        <dbl>
 1 9E                NA           NA             NA           NA
 2 AA                NA           NA             NA           NA
 3 AS                NA           NA             NA           NA
 4 B6                NA           NA             NA           NA
 5 DL                NA           NA             NA           NA
 6 EV                NA           NA             NA           NA
 7 F9                NA           NA             NA           NA
 8 FL                NA           NA             NA           NA
 9 HA               -6.92         4.06           4.90         4.01
10 MQ                NA           NA             NA           NA
11 OO                NA           NA             NA           NA
12 UA                NA           NA             NA           NA
13 US                NA           NA             NA           NA
14 VX                NA           NA             NA           NA
15 WN                NA           NA             NA           NA
16 YV                NA           NA             NA           NA
```

There are NA values that need to be removed

5. Make your `se` function handle NAs with an na.rm option. Test your new function (you can call it `se` again) on a vector that doesn't include NA and on the same vector with an added NA. **Be sure to check that it gives the expected output with na.rm = TRUE and na.rm = FALSE.** Make na.rm = FALSE the default value. Repeat #4. (Hint: be sure when you divide by sample size you don't count any NAs)

```r
SE2 <- function(x) {
sqrt(var(x, na.rm = TRUE)/(sum(!is.na(x))))
  }
```

```r
flights |>
  group_by(carrier) |>
  summarize(mean_arr_delay = mean(arr_delay, na.rm = TRUE),
            se_arr_delay = SE2(arr_delay),
            mean_dep_delay = mean(dep_delay, na.rm = TRUE),
            se_dep_delay = SE2(dep_delay))
```

```
# A tibble: 16 x 5
   carrier mean_arr_delay se_arr_delay mean_dep_delay se_dep_delay
   <chr>            <dbl>        <dbl>          <dbl>        <dbl>
 1 9E               7.38         0.381          16.7         0.348
 2 AA               0.364        0.238           8.59        0.209
 3 AS              -9.93         1.37            5.80        1.18
 4 B6               9.46         0.184          13.0         0.165
 5 DL               1.64         0.203           9.26        0.182
 6 EV              15.8          0.221          20.0         0.205
 7 F9              21.9          2.36           20.2         2.23
 8 FL              20.1          0.960          18.7         0.933
 9 HA              -6.92         4.06            4.90        4.01
10 MQ              10.8          0.273          10.6         0.247
11 OO              11.9          9.02           12.6         8.00
12 UA               3.56         0.170          12.1         0.148
13 US               2.13         0.235           3.78        0.199
14 VX               1.76         0.699          12.9         0.626
15 WN               9.65         0.427          17.7         0.394
16 YV              15.6          2.27           19.0         2.11
```

6. Create `both_na()`, a function that takes two vectors of the same length and returns how many positions have an NA in both vectors. Hint: create two vectors like `test_x <- c(1, 2, 3, NA, NA)` and `test_y <- c(NA, 1, 2, 3, NA)` and write code that works for `test_x` and `test_y`, then turn it into a function that can handle any `x` and `y`. (In this case, the answer would be 1, since both vectors have NA in the 5th position.) Test it for at least one more combination of `x` and `y`.

```r
test_x <- c(1, 2, 3, NA, NA)
test_y <- c(NA, 1, 2, NA, NA)

 sum(is.na(test_x) == "TRUE" & is.na(test_y) == "TRUE")
```

```
[1] 2
```

```
both_na <- function(x, y) {
   sum(is.na(x) == "TRUE" & is.na(y) == "TRUE")
}


both_na(test_x, test_y)
```

```
[1] 2
```

7. Run your code from (6) with the following two vectors: `test_x <- c(1, 2, 3, NA, NA, NA)` and `test_y <- c(NA, 1, 2, 3, NA)`. Did you get the output you wanted or expected? Modify your function using `if`, `else`, and `stop` to print an error if x and y are not the same length. Then test again with `test_x`, `test_y` and the sets of vectors you used in (6).

```
test_x <- c(1, 2, 3, NA, NA, NA)
test_y <- c(NA, 1, 2, 3, NA)

test_x2 <- c(1, 2, 3, NA, NA)
test_y2 <- c(NA, 1, 2, NA, NA)

 sum(is.na(test_x) == "TRUE" & is.na(test_y) == "TRUE")
```

```
Warning in is.na(test_x) == "TRUE" & is.na(test_y) == "TRUE": longer object
length is not a multiple of shorter object length
```

```
[1] 2
```

```
both_na2 <- function(x, y) {
  if (length(x) != length(y)) {
   stop("The length of the vectors must be the same", call. = FALSE) }
   else {sum(is.na(x) == "TRUE" & is.na(y) == "TRUE")}
}


both_na2(test_x, test_y)
```

```
Error:
! The length of the vectors must be the same
```

```
both_na2(test_x2, test_y2)
```

```
[1] 2
```

8. Here is a way to get `not_cancelled` flights in the flights dataset:

```
not_cancelled <- flights |>
  filter(!is.na(dep_delay), !is.na(arr_delay))
```

Is it necessary to check is.na for both departure and arrival? Using summarize, find the number of flights missing departure delay, arrival delay, and both. (Use your new function!)

```
flights |>
  group_by() |>
  summarize(dep_delay_na = sum(is.na(dep_delay)),
            arr_delay_na = sum(is.na(arr_delay)),
            both_na = both_na2(arr_delay, dep_delay))
```

```
# A tibble: 1 x 3
  dep_delay_na arr_delay_na both_na
         <int>        <int>   <int>
1         8255         9430    8255
```

9. Read the code for each of the following three functions, puzzle out what they do, and then brainstorm better names.

```
min_difference <- function(time1, time2) {
  hour1 <- time1 %/% 100
  min1 <- time1 %% 100
  hour2 <- time2 %/% 100
  min2 <- time2 %% 100

  (hour2 - hour1)*60 + (min2 - min1)
}


area_inches <- function(lengthcm, widthcm) {
  (lengthcm / 2.54) * (widthcm / 2.54)
}
```

```
no_response <- function(x) {
  fct_collapse(x, "non answer" = c("No answer", "Refused",
                                   "Don't know", "Not applicable"))
}
```

10. Explain what the following function does and demonstrate by running `foo1(x)` with a few appropriately chosen vectors `x`. (Hint: set x and run the "guts" of the function piece by piece.)

```
foo1 <- function(x) {
  diff <- x[-1] - x[1:(length(x) - 1)]
  sum(diff < 0)
}


x <- c(1, 3, 5, 7)

x[-1]
```

```
[1] 3 5 7
```

```
x[1:(length(x)-1)]
```

```
[1] 1 3 5
```

```
x[-1] - x[1:(length(x)-1)]
```

```
[1] 2 2 2
```

```
foo1(x)
```

```
[1] 0
```

Takes difference of the entries of x without the first entry and the first through 1 minus the length entries. It returns the amount of values that are less than 0.

11. The `foo1()` function doesn't perform well if a vector has missing values. Amend `foo1()` so that it produces a helpful error message and stops if there are any missing values in the input vector. Show that it works with appropriately chosen vectors `x`. Be sure you add `error = TRUE` to your R chunk, or else knitting will fail!

```r
foo1_new <- function(x) {
  if(sum(is.na(x)) == 0)
  {diff <- x[-1] - x[1:(length(x) - 1)]
  sum(diff < 0) }
  else{print("vector cannot contain missing values!")}
}

missing_x <- c(1, 2, 3, NA, 5)
normal_x <- c(1, 3, 4, 5)

foo1_new(normal_x)
```

```
[1] 0
```

```r
foo1_new(missing_x)
```

```
[1] "vector cannot contain missing values!"
```

12. Write a function called `greet` using `if`, `else if`, and `else` to print out "good morning" if it's before 12 PM, "good afternoon" if it's between 12 PM and 5 PM, and "good evening" if it's after 5 PM. Your function should work if you input a time like: `greet(time = "2018-05-03 17:38:01 CDT")` or if you input the current time with `greet(time = Sys.time())`. [Hint: check out the `hour` function in the `lubridate` package]

```r
greet <- function(time = Sys.time()) {

hour_now <- hour(time)

if(hour_now < 12) {print("Good Morning", call. = FALSE)}
  else if(hour_now < 17 & hour_now > 12) {print("Good Afternoon", call. = FALSE)}
  else{print("Good Evening", call. = FALSE)}
}

greet("2026-10-22 18:55:50 CDT")
```

```
[1] "Good Evening"
```

```r
greet("2026-10-22 09:55:50 CDT")
```

```
[1] "Good Morning"
```

```
greet("2026-10-22 14:55:50 CDT")
```

```
[1] "Good Afternoon"
```

13. Modify the `summary6()` function from earlier to add an argument that gives the user an option to remove missing values, if any exist. Show that your function works for (a) the `hwy` variable in `mpg_tbl <- as_tibble(mpg)`, and (b) the `age` variable in `gss_cat`.

```
summary6na <- function(data, var, na_remove = FALSE) {
  data |>
    summarize(
    mean = mean({{ var }}, na.rm = na_remove),
    median = median({{ var }}, na.rm = na_remove),
    sd = sd({{ var }}, na.rm = na_remove),
    IQR = IQR({{ var }}, na.rm = na_remove),
    n = n(),
    n_miss = sum(is.na({{ var }})),
    .groups = "drop"    # to leave the data in an ungrouped state
  )
}

mpg |> summary6na(hwy, na_remove = TRUE)
```

```
# A tibble: 1 x 6
   mean median    sd   IQR     n n_miss
  <dbl>  <dbl> <dbl> <dbl> <int>  <int>
1  23.4     24  5.95     9   234      0
```

```
gss_cat |> summary6na(age, na_remove = TRUE)
```

```
# A tibble: 1 x 6
   mean median    sd   IQR     n n_miss
  <dbl>  <int> <dbl> <dbl> <int>  <int>
1  47.2     46  17.3    26 21483     76
```

14. Add an argument to (13) to produce summary statistics by group for a second variable (you should now have 4 possible inputs to your function). Show that your function works for (a) the `hwy` variable in `mpg_tbl <- as_tibble(mpg)` grouped by `drv`, and (b) the `age` variable in `gss_cat` grouped by `partyid`.

```r
summary6na_group <- function(data, group, var, na_remove = FALSE) {
  data |>
    group_by(pick({{group}})) |>
    summarize(
    mean = mean({{ var }}, na.rm = na_remove),
    median = median({{ var }}, na.rm = na_remove),
    sd = sd({{ var }}, na.rm = na_remove),
    IQR = IQR({{ var }}, na.rm = na_remove),
    n = n(),
    n_miss = sum(is.na({{ var }})),
    .groups = "drop"    # to leave the data in an ungrouped state
  )
}

mpg |> summary6na_group(hwy, group = drv, na_remove = TRUE)
```

```
# A tibble: 3 x 7
  drv    mean median    sd   IQR     n n_miss
  <chr> <dbl>  <dbl> <dbl> <dbl> <int>  <int>
1 4      19.2     18  4.08     5   103      0
2 f      28.2     28  4.21     3   106      0
3 r      21       21  3.66     7    25      0
```

```r
gss_tibble <- gss_cat |> summary6na_group(age, group = partyid, na_remove = TRUE)
```

15. Create a function that has a vector as the input and returns the last value. (Note: Be sure to use a name that does not write over an existing function!)

```r
last_val <- function(x) {
  x[length(x)]
}

last_val(x)
```

```
[1] 7
```

16. Save your final table from (14) and write a function to draw a scatterplot of a measure of center (mean or median - user can choose) vs. a measure of spread (sd or IQR - user can choose), with points sized by sample size, to see if there is constant variance. Each point should be labeled with partyid, and the plot title should reflect the variables chosen by the user.

Hint: start with a ggplot with no user input, and then functionize:

```
library(ggrepel)

check_equal_var <- function(data, center, spread, label) {

my_title <- rlang::englue("Plot of {{center}} and {{spread}}")

data |>
  ggplot(aes(x = {{center}}, y = {{spread}})) +
    geom_point(aes(size = n)) +
    geom_smooth(method = lm) +
    geom_label_repel(aes(label = {{label}})) +
    labs(title = my_title)
}

check_equal_var(gss_tibble, median, IQR, partyid)
```

```
`geom_smooth()` using formula = 'y ~ x'
```

